

Programmieren (T3INF1004)

DHBW Stuttgart

Christian Bader

christian.Bader@lehre.dhbw-stuttgart.de

Christian Alexander Holz

christian.holz@lehre.dhbw-stuttgart.de



Recap / Vorstellungen

Live

Kapitelübersicht

- 1 Einführung
- 2 Objektorientierung
- 3 Vertiefung C++
- 4 Die Standard Template Library (STL)
- 5 Clean Code
- 6 Test-driven Development

Kapitel 2: Objektorientierung

1. Grundlagen
2. Header Files
3. Vererbung
- 4. Polymorphismus (Einschub Pointer und Referenzen)**

Exkurs: Git

Visualisierung auf Speicherebene

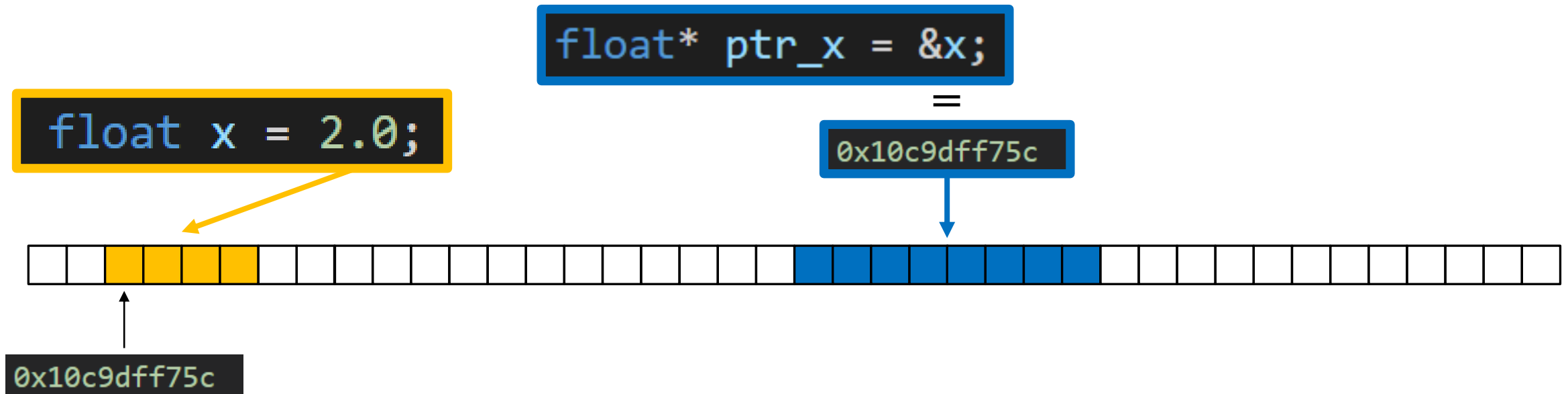
00000000	00000001	00000002	00000003	00000004	00000005	00000006	00000007	00000008

...

- Ausschnitt aus Speicherbereich (RAM)
- Jeder Speicherblock (ein Byte = 8 Bit) hat eine eindeutige Adresse

Visualisierung auf Speicherebene

- Objekt vom Type `float` (4 Byte = 32 Bit) wird hinter Adresse `0x10c9dff75c` gespeichert
- Diese Adresse (64 Bit auf 64 Bit Maschine, also 8 Byte) wird im Pointer `ptr_x` gespeichert. Der Pointer selbst hat wieder eine eigene Adresse.



C++: Referenzen

C++: Pass by reference / pass by value: Speed-up

```
2  int main()
3  {
4      SomeBigObject x;
5      foo(x)
6
7      return 0;
8  }
9
10 void foo(SomeBigObject x)
11 {
12     // do something with Object
13 }
```

Objekt wird kopiert!

```
33  int main()
34  {
35      SomeBigObject x;
36      foo(x)
37
38      return 0;
39  }
40
41  void foo(SomeBigObject& x)
42  {
43      // do something with Object
44      x.getPrivateValues();
45  }
```

Adresse von Objekt wird übergeben!

Vergleich: Pass by reference C/C++

```
void foo(SomeBigObject* x)
```

C:

- Pointer wird übergeben
- Vor Aufruf von Funktion: Referenzierung
- In Funktion: Dereferenzierung oder direkt mit pointer auf Objekt arbeiten
SomeBigObject->doSomething();
- Nicht Nullpointer-safe!

```
void foo(SomeBigObject& x)
```

C++:

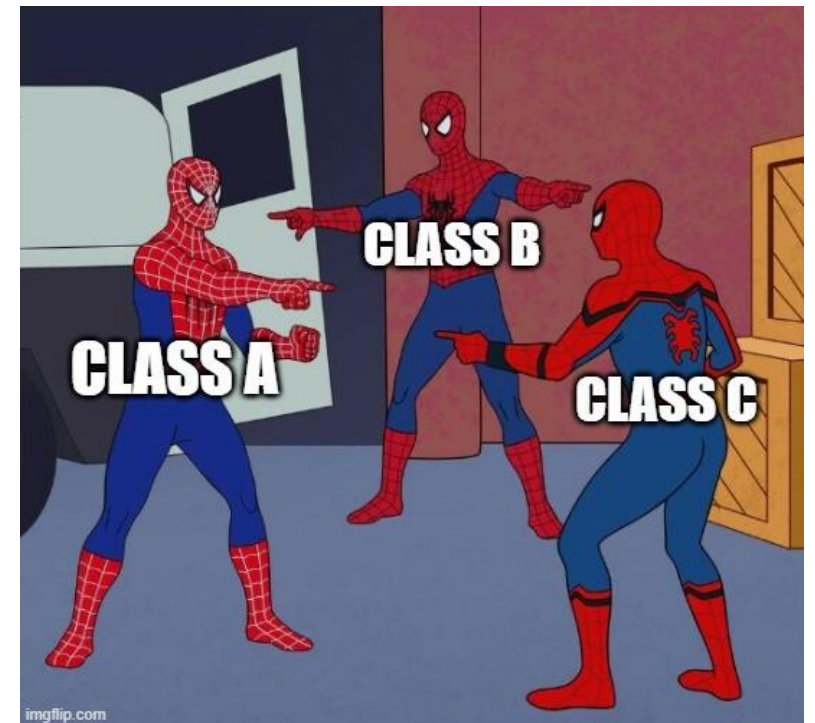
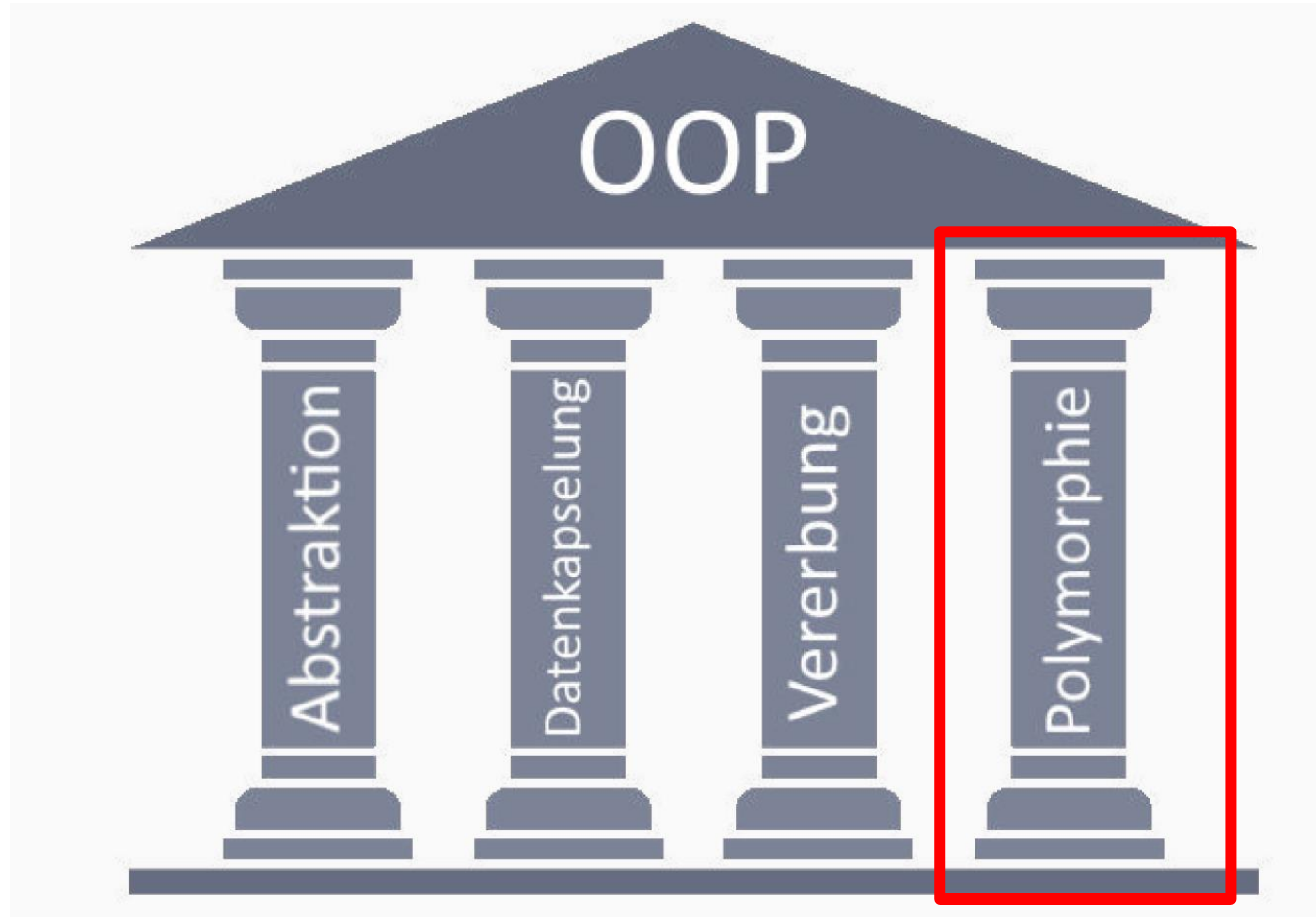
- & legt fest, dass Referenz übergeben wird.
- Vor Aufruf von Funktion: Nichts.
- In Funktion: Nichts
(SomeBigObject.doSomething();)
- Nullpointer-safe!

Kapitel 2: Objektorientierung

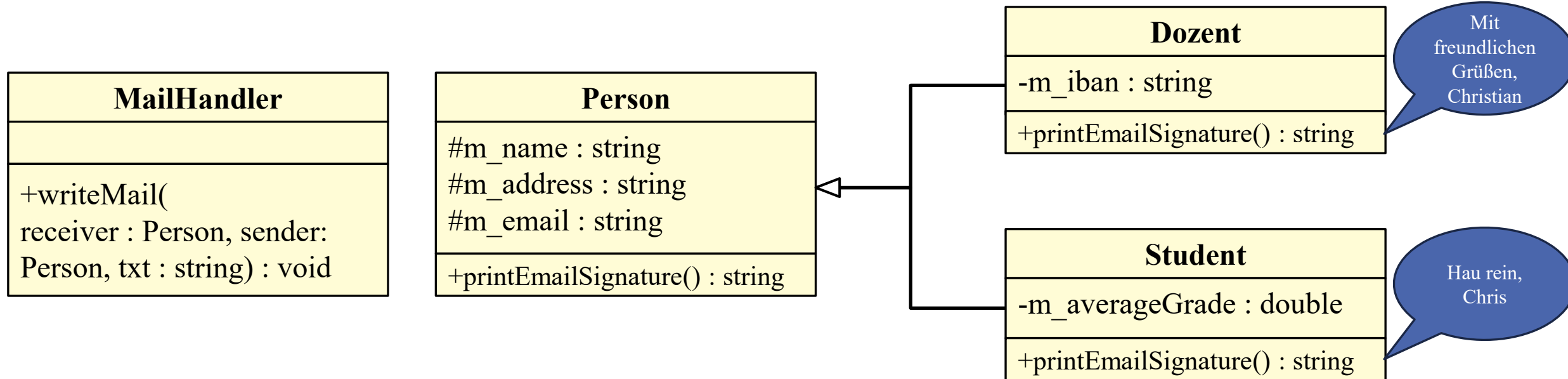
1. Grundlagen
2. Header Files
3. Vererbung
- 4. Polymorphismus**

Exkurs: Git

Die vier Säulen der objektorientierten Programmierung

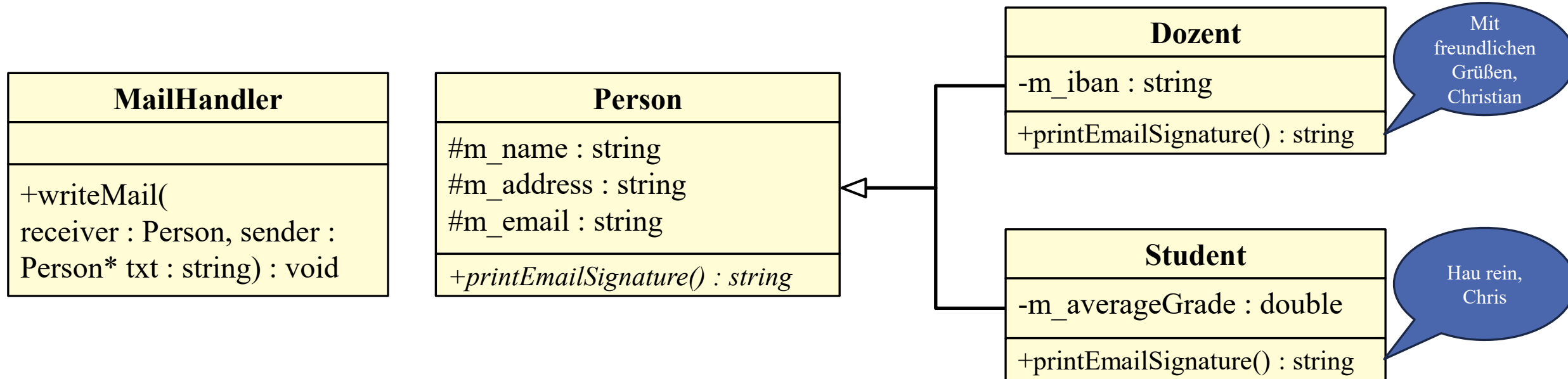


Interfaces mit Vererbung



Polymorphism

Interfaces mit Polymorphismus



Polymorphism

Wichtigster Vorteil von Vererbung: Polymorphismus

■ Definition

- Objekte mit der selben Basis-Klasse haben unterschiedliches Verhalten
- Konkrete Implementierung von Methode nicht in Basis-Klasse sondern kann in den Kind-Klassen überschrieben werden.

■ Vorteile

- Wir können mit Objekten arbeiten von denen wir lediglich den Basis-Typ kennen
- Ermöglicht Erstellen von Interfaces, die festlegen wie etwas genutzt wird (also welche Funktionen mit welcher Signatur `public` sind), aber nicht wie diese genau implementiert werden.
- Damit entsteht eine „**Dependency-Firewall**“: Alles was unserer Klassen benutzt ist nur abhängig von der Interface-Basis-Klasse aber **komplett unabhängig** von der konkreten Implementierung der Methoden
- Zudem kann dadurch eine neue Abstraktionsebene geschaffen werden → Komplexitäts Reduzierung

Keyword: virtual

- Ermöglicht Polymorphismus zur Laufzeit

Vorteile

- Dynamische Bindung
 - Methodenaufruf abhängig vom tatsächlichen Objekttyp
 - Überschreibung von Methode in Unterklassen
-
- =0 → Rein virtuelle Methode, Muss von Unterklassen implementiert werden
→ Abstrakte Klasse

```
// Base class
class Shape
{
public:
    Shape(double width, double height)
        : m_width(width), m_height(height)
    {
    }

    virtual ~Shape()
    {
    }

    virtual double getArea() = 0;
```

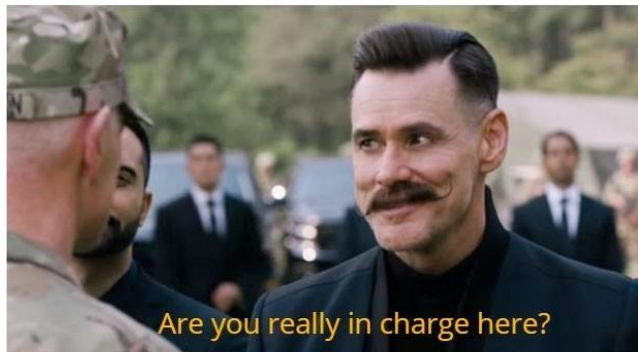
Keyword: override

- Seit C++11 wird `override` unterstützt
- Gibt an, dass diese Funktion eine andere überlädt

Vorteile

- Bessere Lesbarkeit
- Compiler Error, falls keine Überladung stattfindet

When I override my
parent's methods



```
63 // Derived class
64 class Triangle : public Shape
65 {
66 public:
67     // Leave out the constructor for the first try
68     Triangle(double width, double height)
69     : Shape(width, height)
70     {}
71
72     double getArea() override
73     {
74         return (m_width * m_height * 0.5);
75     }
```

Aufgaben: Shapes Polymorph

Verwenden Sie die Aufgabenbeschreibung aus dem Git Repo:
**2_Objektorientierung/234_task_shapes/
TASK_SHAPES_POLYMORPHISMUS.md**

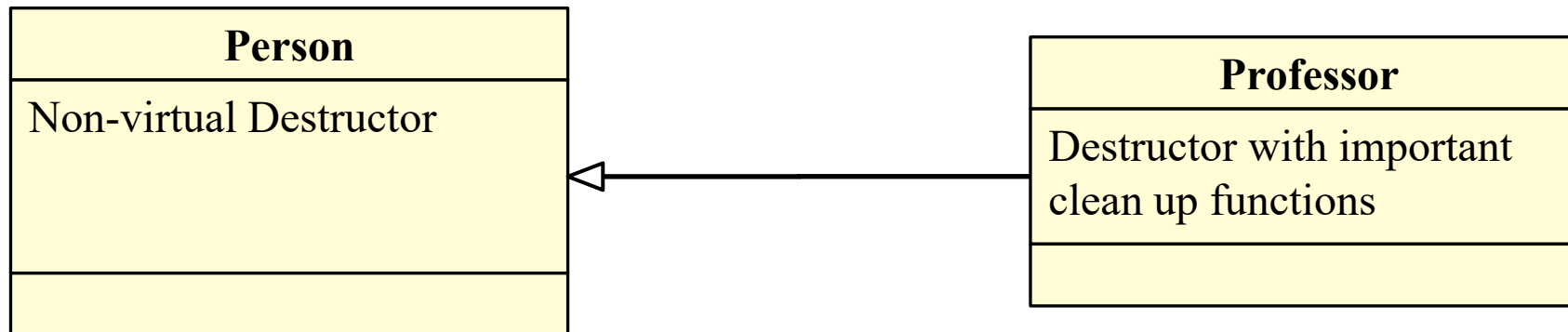
Polymorphism

Polymorphismus: virtueller Destruktor

Destruktoren und Vererbung

Regel: Sobald von einer Klasse geerbt wird muss **Destruktor** in Basis Klasse **virtuell** sein!

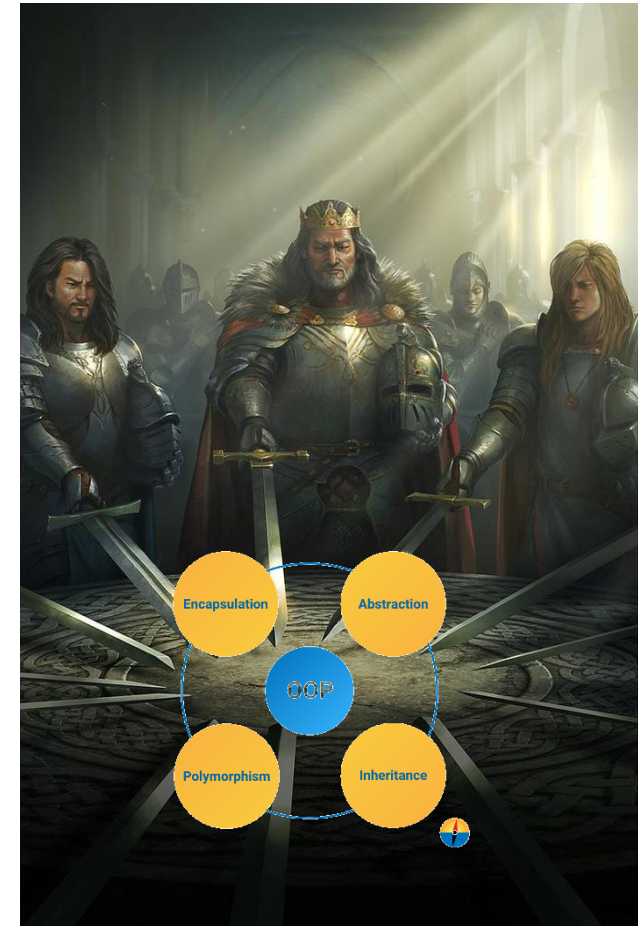
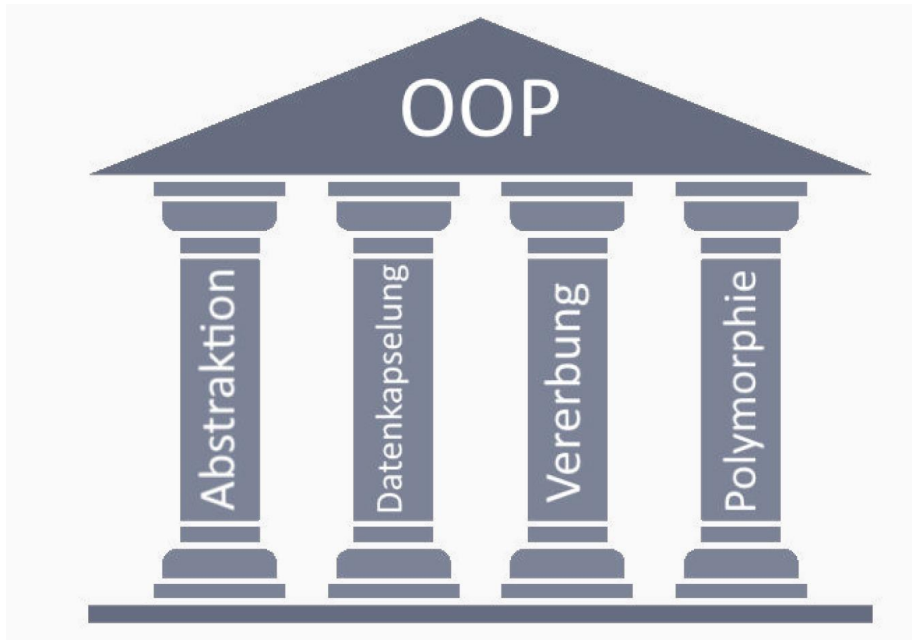
- Sonst:



- Destruktor von Person wird nicht überschrieben und das clean-up funktioniert nicht so wie gehofft → Vermeidbare Fehlerquelle!
- Am einfachsten: **Alle Destruktoren virtuell!**

```
virtual ~Person(){};
```

Recap: Alle Säulen werden genutzt!



Aufgaben: Shapes Polymorph

Verwenden Sie die Aufgabenbeschreibung aus dem Git Repo:

**2_Objektorientierung/235_task_animal/
TASK_ANIMAL_POLYMORPHISMUS.md**